

Product Requirements Document (PRD)

Project Name

Smart Crime Analytics & Police Investigation System

Subject

Data Structures and Algorithms (DSA) using C++

Project Type

Console-Based Application

Project Objective

Develop a C++ console application that demonstrates practical implementation of Data Structures and Algorithms in a crime analytics and police investigation environment.

The project must strictly implement all requirements mentioned in the case study and must not include any additional features.

Required DSA Concepts

1. Sorting Algorithms
2. Searching Algorithms
3. Stack
4. Queue
5. Binary Search Tree (BST)
6. AVL Tree
7. Hashing

8. Breadth First Search (BFS)

9. Depth First Search (DFS)

Data Models

Crime Case

Fields:

- Case ID
 - Crime Type
 - Severity Level
 - Investigation Status
-

Criminal Record

Fields:

- Criminal ID
 - Criminal Name
 - Crime Type
-

Crime Report

Fields:

- Report ID
 - Crime Type
-

Investigation Action

Fields:

- Action Description
-

Functional Modules

Module 1: Crime Case Management

Functions:

- Add Crime Case
- View Crime Cases

Purpose:

Manage crime case records.

Module 2: Sorting Module

Functions:

- Sort Cases by Severity

DSA:

- Bubble Sort

Purpose:

Rank crime cases based on severity.

Module 3: Searching Module

Functions:

- Search Crime Case by ID
- Search Criminal by ID

DSA:

- Linear Search

Purpose:

Retrieve case and criminal records.

Module 4: Queue Module

Functions:

- Add Crime Report
- Process Crime Report
- View Pending Reports

DSA:

- Queue

Purpose:

Process crime reports using FIFO.

Module 5: Stack Module

Functions:

- Add Investigation Action
- Undo Last Action
- View Investigation History

DSA:

- Stack

Purpose:

Manage investigation rollback procedures.

Module 6: BST Module

Functions:

- Insert Criminal
- Search Criminal
- Display Criminal Records

DSA:

- Binary Search Tree

Purpose:

Store criminal records.

Module 7: AVL Module

Functions:

- Insert Criminal
- Search Criminal
- Display Criminal Records

DSA:

- AVL Tree

Purpose:

Store criminal records using balanced trees.

Module 8: Hashing Module

Functions:

- Insert Criminal
- Search Criminal

DSA:

- Hashing

Purpose:

Rapid suspect identification.

Module 9: Criminal Network Module

Functions:

- Add Connection

- BFS Traversal
- DFS Traversal

DSA:

- Graph
- BFS
- DFS

Purpose:

Analyze criminal networks.

Module 10: Crime Analytics Module

Functions:

- Total Cases
- Open Cases
- Closed Cases
- Highest Severity Crime
- Most Frequent Crime

Purpose:

Generate crime analytics, crime patterns and predictive insights.

Dummy Data Requirements

System must start with predefined sample data.

Dummy data must be stored separately from DSA logic.

Dummy data required for:

- Crime Cases
- Criminal Records
- Crime Reports

- Criminal Network Connections

Users must also be able to add additional data during runtime.

Code Requirements

- Simple beginner-friendly C++
- No frontend
- No extra features
- No advanced libraries
- No complex architecture
- Detailed comments in every module
- Detailed comments in every function
- DSA sections clearly labeled
- Easy to explain during viva